

ICS/OT Protocol Analysis — Student Worksheet

Work these exercises with the two teaching captures open in Wireshark (`dnp3_substation.pcap`, `mqtt_iiot_telemetry.pcap`) and the Docker lab running. Each exercise lists the steps, then the question to answer. Record your answers in the space after each question.

Student name / date: _____

DNP3 — DNP3

Capture: `dnp3_substation.pcap`

Q1. Find the control lifecycle

1. Open `dnp3_substation.pcap` in Wireshark.
2. Apply the display filter `dnp3.al.func` in {3,4,5} to isolate every control.
3. Identify the legitimate SELECT—OPERATE pair and the rogue DIRECT OPERATE.

Question. How many control messages are there, and which one lacks a SELECT — and why does that matter?

Answer:

Q2. Spot the impostor by address

1. Filter `dnp3` and add columns for `ip.src` and `dnp3.src` (DNP3 link source).
2. Compare the IP source and the DNP3 link source for every control frame.

Question. What is inconsistent about frame 27, and which field did the attacker forge?

Answer:

Q3. Turn packets into detections with ICSNPP

1. In the lab container run: `zeek -C -r /pcaps/dnp3_substation.pcap icsnpp-dnp3`
2. Open `dnp3_control.log` and `dnp3_objects.log`.

Question. Which single log line is your best alert for the attack, and what field makes it detectable?

Answer:

Q4. Design the control

1. Re-read security findings D1 and D3.
2. Given a substation you cannot re-flash to add DNP3-SA tomorrow, list compensating controls you can deploy this week.

Question. Name three compensating controls that reduce the frame-27 risk without changing the outstation firmware.

Answer:

MQTT — MQTT

Capture: mqtt_iot_telemetry.pcap

Q5. Read a password off the wire

1. Open mqtt_iot_telemetry.pcap in Wireshark.
2. Apply mqtt.msgtype==1 and expand the CONNECT tree on frame 4.

Question. What are the HMI's username and password, and which single control would have prevented you from reading them?

Answer:

Q6. Trace one message to two subscribers

1. Filter mqtt.msgtype==3 (all PUBLISH).
2. Follow the third telemetry reading from the sensor through the broker.

Question. Starting at the sensor's publish (frame 46), which frames deliver that same reading, and to whom?

Answer:

Q7. Catch the anonymous intruder

1. Filter mqtt.msgtype==1 and compare the connect flags of frames 4, 15, and 38.
2. In the lab container run: `zeek -C -r /pcaps/mqtt_iot_telemetry.pcap` and open mqtt_connect.log.

Question. Which CONNECT is anonymous, and what does mqtt_connect.log show for it?

Answer:

Q8. Harden the broker

1. In the lab, edit mosquitto.conf: set allow_anonymous false, add a password_file, and an acl_file scoping the HMI to read plant/+/telemetry only.
2. Restart the broker and re-run the publisher/subscriber and a '#' subscriber.

Question. After hardening, what happens to the anonymous connect, and how does the capture differ?

Answer:

Synthesis

Q9.

A classmate reduces both intrusions to one root cause — 'neither protocol authenticates, so the rogue host is trusted.' That is only half right. Using the captures, separate AUTHENTICATION (who are you?) from AUTHORIZATION (are you allowed to do this?). For DNP3 frame 27 and

for the MQTT command injection, state (i) whether authentication happened at all, (ii) whether the failure is one of authentication or authorization, and (iii) the exact missing mechanism that would have stopped it. Cite the specific frame(s).

Answer:

Q10.

For each protocol, name the standard/control that adds authentication and say whether it also adds confidentiality.

Answer:

Q11.

You can monitor but not immediately re-engineer these systems. Give one Zeek/ICSNPP-based detection for each capture.

Answer: